ICS441 Lab1 : Risk Assessment & Register Development

Course: ICS 441 - Advanced Cybersecurity and Ethical Hacking

2. Scenario: "VulnFeed" Social Media Startup

3. Vulnerability Reports (Expanded List)

Critical Vulnerabilities:

Vulnerability 1: Cloud Storage Misconfiguration

- **Description:** A Google Cloud Storage bucket named `vulnfeed-user-uploads` is configured to allow public read access. This bucket contains profile pictures and uploaded media. The testers were able to list and download all files without any authentication.
- **Technical Detail:** The bucket's IAM permissions contain the assignment `allUsers:roles/storage.objectViewer`.

Vulnerability 2: SQL Injection in Search

- **Description:** The main search feature on the VulnFeed homepage is vulnerable to SQL Injection. The testers extracted the entire list of usernames, emails, and hashed passwords by injecting a simple UNION SELECT payload.
- **Technical Detail:** The vulnerable parameter is `?q=` in the `GET /search` request. The backend directly concatenates user input into the SQL query.

Vulnerability 3: Broken Access Control on API

- **Description:** The API endpoint `GET /api/users/{user_id}/posts` does not check authorization. An attacker can change the `user_id` parameter to view the private posts of any user, even if their account is set to private.
- **Technical Detail:** The API uses the user ID from the session token for display but uses the user ID from the URL parameter for the database query, with no check to see if they match.

New - Vulnerability 4: Use of Hardcoded API Keys

- **Description:** The front-end JavaScript code contains a hardcoded API key for a third-party service (e.g., SendGrid for email or a Maps API). This key is exposed to every user's browser and has already been abused, running up a large bill.
- **Technical Detail:** The key is found in a publicly accessible `config.js` file: `const SENDGRID_API_KEY = 'SG.sk_12345...';`

New - Vulnerability 5: Missing Multi-Factor Authentication (MFA)

- **Description:** The application does not offer Multi-Factor Authentication (MFA) for any users, including administrators. The recent pen test showed that an attacker who phishes a user's password has full, unrestricted access to their account and data.
- **Technical Detail:** The `/login` endpoint only accepts a username and password. There is no endpoint or flow for a second authentication factor.

New - Vulnerability 6: Weak Password Policy

- **Description:** The application's password policy only requires a minimum of 5 characters with no complexity requirements. This makes brute-force attacks and credential stuffing highly effective.
- **Technical Detail:** The password validation regex is `/.{5,}/`.

4. Lab Tasks (Group Work) - Enhanced

1. **Create a Blank Risk Register:** Create a table with columns for: Risk ID, Title, Description, Likelihood (1-5), Impact (1-5), Inherent Risk Score, Mitigation, Owner, Due Date, Status.
2. **Analyze & Prioritize ALL SIX Vulnerabilities:**
    - For each vulnerability, assign **Likelihood** and **Impact** scores.
    - **Calculate the Risk Score** (L x I).
    - **Prioritize** all six vulnerabilities from highest to lowest risk.
3. **Propose Mitigations:** For each vulnerability, write a clear, technical recommendation for a fix.
4. **Assign Ownership & Deadlines:** Assign each fix to a team (Backend, Frontend, DevOps, Security) and set a realistic deadline based on priority.
5. **Be Prepared to Defend Your Choices:** The instructor (CTO) will ask why one vulnerability was ranked higher than another.

5. Lab Deliverable

Each group must submit **one PDF document** containing:

1. **Completed Risk Register:** A filled-out table with entries for all **six** vulnerabilities.
2. **Prioritized List:** A numbered list (1 being highest priority) of the six vulnerabilities.
3. **Rationale Paragraph (1 page max):** Explain the reasoning behind the top **two** highest priority risks and the bottom **one** lowest priority risk. Justify your scores and ranking.